Homework 1: Kinematics and Control

16-662: Robot Autonomy

Spring 2026

Due: Monday, Feb 09, 2026 at 11:59 PM ET

Instructor: Oliver Kroemer

Teaching Assistants: Sreeharsha Paruchuri, Abhinandan Vellanki

Introduction

This homework will focus on the topics of kinematics and control from the lectures. You will explore force and impedance control on a simple scene in the MuJoCo (**Mu**lti-**Jo**int Dynamics with **Co**ntact) simulator for the first part, and then implement forward and inverse kinematics for the Franka arm. To set up MuJoCo for the homeworks instructions have been provided in the next section. An additional introductory pdf has been provided to get you started with MuJoCo once you have set up the simulator. The homeworks have been tested on Ubuntu. If you require a virtual machine or help with the installation, please let me know. You are encouraged to work together and discuss questions with your peers, but please write your own code for the implementations.

AI Usage Policy

The use of AI to generate code for this homework is **strictly forbidden**. You are instead recommended to attend the Teaching Assistants' office hours or research by yourself or with your peers.

Part I: MuJoCo Setup

To set up MuJoCo, follow the steps below:

1. Navigate to your desired working directory.
2. Create a separate conda / Python environment to run your homework. To do this, run

   ```
   conda create -name <env_name>
   ```
   OR
   ```
   python3 -m venv <env_name>
   ```

   Replace `<env_name>` with a name of your choice.
3. Activate the environment by running

```
conda activate <env_name>
```
OR
```
source <env_name>/bin/activate
```

4. If you are using a conda environment, install `pip` by running

```
conda install pip
```

5. Locate where `pip` was installed for your conda environment. This path should look similar to

```
/anaconda/envs/<env_name>/
```

   If you are using a Python environment, the `pip` should already exist. You can verify this by running `which pip` which should show the following path: `<env_name>/bin/pip`

6. Use this executable to install MuJoCo:

```
/anaconda/envs/<env_name>/bin/pip install mujoco
```

   Inside Python environments, you can simply run `pip install mujoco`

   This installs both the Python bindings and a local copy of the MuJoCo library, so no separate MuJoCo installation is required.

   IMPORTANT: Ensure there are no whitespaces in your folder names as this can cause problems while running the MuJoCo executable, especially when working inside Python environments.

   MacOS: To run Python scripts that use MuJoCo, use the mjpython binary to run them, i.e., replace the usual `python script.py` with `mjpython script.py`

Part II: PID controller

In this section, you will implement PID controllers in a simple MuJoCo scene to familiarize yourself with the simulation environment and understand the basics of PID control.

You will find two scripts located in the `control` folder: `PandaArmControl_Part1.py` and `PandaArmControl_Part2.py`. The primary difference between these files lies in the world being loaded. In Part 1, the environment contains the Franka arm and a static whiteboard, while in Part 2, the Franka arm interacts with a moving whiteboard.

2.1 Part 1

In Part 1, you are expected to implement an impedance controller and a force controller such that the arm generates **15 N** of force against the whiteboard, as shown in Figure 1.

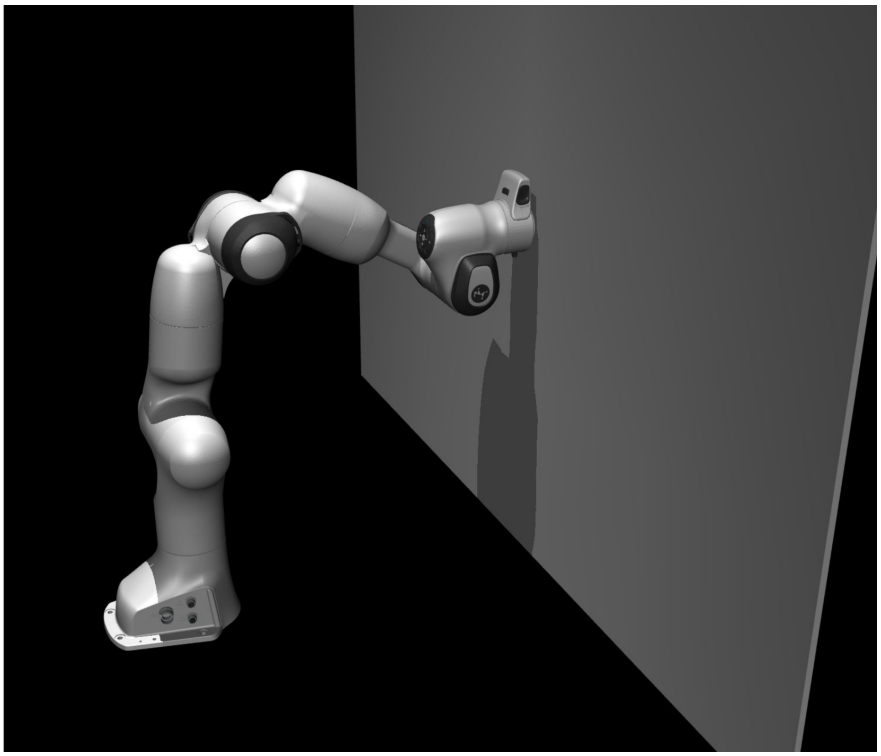

Figure 1: Franka arm applying force against a static whiteboard in MuJoCo.

To simplify things, the robot is initialized with a predefined set of joint positions. This initial state is stored in the MJCF model file and is loaded each time the simulator is launched. You are encouraged to review the script to understand how the control pipeline is structured. Briefly:

1. The `main` function initializes the `mjModel` and `mjData` data structures, sets the relevant keyframe for the homework, and registers the control callback function.

2. The control callbacks are defined above the `main` function. You are encouraged to experiment with gravity compensation and position control to better understand how the robot interacts with the environment. For more details on interacting with the simulator, refer to the Intro to MuJoCo PDF provided with this handout.

2.2 Part 2

In this section, you will use the same controllers implemented in Part 1, but introduce disturbances in the form of a sinusoidally moving whiteboard. Copy your controller implementations from Part 1 into `PandaArmControl_Part2.py` and run the file. Observe and analyze how the robot behaves under this disturbance.

2.3 Plotting for Part 1 and Part 2

Both scripts will output CSV files containing time and force values recorded over a 10-second simulation window. Ensure that the simulator is run for at least 10 seconds for each controller (Impedance and Force control) in both Part 1 and Part 2.

- You may use any plotting tool of your choice to visualize force versus time. Be sure that each plot includes a title, axis labels, and a legend.

Submission

For the control section, submit a single PDF page containing the following items:

- A screenshot of the force plots for the static whiteboard case. Plot the outputs of the force controller and impedance controller superimposed on the same figure. Ensure the plot includes a title, axis labels, and a legend clearly identifying each controller.
- A screenshot of the force plots for the oscillating whiteboard case. Plot the outputs of the force controller and impedance controller superimposed on the same figure, again including a title, axis labels, and a legend.
- A screenshot of the robot when the whiteboard is near its maximum amplitude (i.e., closest to the robot).
- A brief (2–3 sentence) explanation describing the differences in behavior between the force controller and the impedance controller.
- A brief (2–3 sentence) overview of your observation and analysis on how the robot behaves under this disturbance.

Part III: Kinematics

3.1 Forward Kinematics

In this section, you will implement the forward kinematics (FK) for the Franka arm and compute the Jacobian. Please use Python for your implementation. Starter code has been provided in `Franka.py` and `RobotUtil.py` in the `Kinematics` folder. You may use the helper functions in `RobotUtil.py` or implement your own as needed.

Your task is to implement the computation of the forward kinematics and Jacobian in the `ForwardKin()` method of the `FrankaArm` class in `Franka.py`. Your FK method should output the transformation matrices for each joint based on the input joint angles, as well as the Jacobian matrix.

To test your implementation, run your FK method for the following joint configurations and compute the resulting end-effector poses. Submit your computed end-effector poses for each configuration listed below. (Note: these joint configurations are also defined as a list named `joint_targets` in `MainTest.py`.)

Targets:

- $\mathbf{q}_1 = [0°,\ 0°,\ 0°,\ 0°,\ 0°,\ 0°,\ 0°]$
- $\mathbf{q}_2 = [0°,\ 0°,\ -45°,\ -15°,\ 20°,\ 15°,\ -75°]$
- $\mathbf{q}_3 = [0°,\ 0°,\ 30°,\ -60°,\ -65°,\ 45°,\ 0°]$

Expected output from FK for $\mathbf{q}_1$:

$$\mathbf{t} = \begin{bmatrix} 0.088 \\ 0 \\ 0.926 \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

3.2 Inverse Kinematics

In this section, you will implement the inverse kinematics (IK) for the Franka arm using the damped least-squares method. For the weighting matrices, use the following definitions for $\mathbf{C}$ and $\mathbf{W}$:

$$\mathbf{C} = \begin{bmatrix} 1000000.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1000000.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1000000.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1000.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1000.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1000.0 \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 100.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 100.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 100.0 \end{bmatrix}$$

Please use Python for your implementation. Write your code in the IK method `IterInvKin()` of the `FrankaArm` class in `Franka.py`.

Test your IK implementation: Run your IK method using the starting joint configuration and target end-effector pose defined in `Main.py` (referred to as `qInit` and `HGoal` in the code, respectively), and compute the joint configuration that achieves the desired end-effector pose. **Note: the $\mathbf{q}_{\text{init}}$ given below is in radians.**

$$\mathbf{q}_{\text{init}} = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ -2.11 \\ 0.0 \\ 3.65 \\ -0.785 \end{bmatrix} \qquad \mathbf{t}_g = \begin{bmatrix} 0.6 \\ 0 \\ 0.5 \end{bmatrix} \qquad \mathbf{R}_g = \begin{bmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & 1.0 & 0.0 \\ -1.0 & 0.0 & 0.0 \end{bmatrix}$$

3.3 Submission

For the kinematics section, submit a single PDF page containing the following items:

- The three end-effector poses (in matrix form) corresponding to $\mathbf{q}_1$, $\mathbf{q}_2$, and $\mathbf{q}_3$.
- The final joint angles that achieve the target end-effector pose $(\mathbf{R}_g, \mathbf{t}_g)$.
- Use the position controller callback to move the robot to the joint configuration returned by your inverse kinematics solver, and include an image of the robot at this configuration.

That's it! You have completed HW 1! =D